ÍNDICE

1 Conceptos a repasar.....1
2 Programando procesos 2

1 Conceptos a repasar

- Programa (código fuente, ejecutable) vs Proceso.
- Programa ejecutado vs interpretado. El comando ps identifica el ejecutable y el intérprete respectivamente.
- Servicios y demonios.
- Equipos con un/varios procesadores y un/varios núcleos.
- Sistemas monotarea vs multitarea. Programación concurrente.
- Programación paralela.
- Programación distribuida.
- Contexto de un proceso. Bloque de control de proceso (BCP). Pid, contador de programa, imagen de memoria, pila de ejecución, estado del proceso.
- Estados de un proceso. Nuevo, listo, en ejecución, bloqueado, finalizado.
- Planificador de procesos a largo y corto plazo. Describir actividad de intercambio de procesos por el planificador (cambio de contexto).
- Tipos de planificación a corto plazo
    - Sin desalojo o cooperativa.
    - Apropiativa.
    - Tiemplo compartido.
- Tipos de algoritmos. FCFS (normalmente no apropiativo), SJF (sin desalojo o apropiativo), RR (apropiativa), por prioridad (apropiativa).
- Comunicación entre procesos. Sockets, flujos de entrada y salida, RPC (Remote Process Protocol), persistencia (ficheros, bases de datos …).
- Gestión de recursos compartidos, sincronización, secciones críticas, exclusión mutua, interbloqueo, inanición.
- Operaciones atómicas.

2 Programando procesos

ProcessBuilder y lectura de Salida del proceso hijo capturada en el proceso padre

```
import java.io.BufferedReader;
import java.io.File;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;

public class Tema1Prueba {
	public static void main (String [] args)
	{
		String [] sArg = new String [2];
		sArg[0] = "/usr/java/jdk-20.0.2/bin/java"; // Consultar localizacion con which java
		sArg[1] = "Tema1PruebaH";                                  // .class, sin indicar
		ProcessBuilder mipb = null;
		Process miProc = null;
		mipb = new ProcessBuilder(sArg);
		// mipb = new ProcessBuilder("/usr/java/jdk-20.0.2/bin/java","Tema1PruebaH");
		// mipb = new ProcessBuilder("python3","prueba.py");
		// mipb = new ProcessBuilder("ls","-la");
		// mipb = new ProcessBuilder("/usr/java/jdk-20.0.2/bin/java","Tema1PruebaHe");
		// mipb = new ProcessBuilder("firefox");
		mipb.directory(new File("./bin"));

		try {
			miProc = mipb.start();		// Inicia el proceso
		} catch (IOException e) {
			e.printStackTrace();
		}

		System.out.println(miProc.pid());		// Pid del proceso iniciado

		// Enlazar salida estandar del proceso hijo con este proceso padre
		InputStream mils = miProc.getInputStream();
		InputStreamReader milsr = new InputStreamReader(mils);
		BufferedReader milsrb = new BufferedReader(milsr);
		String sInfo = "";
		try {
			while((sInfo = milsrb.readLine()) != null)
			{
				System.out.println(sInfo);
			}
		} catch (IOException e) {
			// TODO Auto-generated catch block
			e.printStackTrace();
		}
	}
}

public class Tema1PruebaH {
	public static void main (String [] args)
	{
		System.out.println("HOLA DAM");
		System.out.println("HOLA DAM-2");
	}
}
```

Fichero prueba.py Lo pongo en ./bin del proyecto que es la ruta de inicio indicada en le objeto ProcessBuilder con el método directory.

```
import sys
print("HOLA DESDE Python")
sys.exit(0)
```

Runtime y lectura de Salida del proceso hijo capturada en el proceso padre

```
import java.io.BufferedReader;
import java.io.File;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;

public class Tema1Prueba {
	public static void main (String [] args)
	{
		// No existe la entrada directory de ProcessBuilder. Poner ejecutables en la ruta del proyecto
		String [] sArg = new String [2];
		sArg[0] = "/usr/java/jdk-21.0.4/bin/java"; // Consultar localizacion con which java
		sArg[1] = "Tema1PruebaH";                                        // .class, sin indicar
		// sArg[0] = "python3";
		// sArg[1] = "prueba.py";

		// String [] sArg = new String [1];
		// sArg[0] = "firefox";
		Process miProc = null;

		try {
			// El proceso hijo se debe encontrar en la ruta del proyecto
			// miProc = Runtime.getRuntime().exec(sArg);
			// Equivalente al metodo directory de proccesBuilder. Se puede indicar la ruta del proc hijo
			miProc = Runtime.getRuntime().exec(sArg,null, new File("./bin"));

		} catch (IOException e) {
			e.printStackTrace();
		}

		System.out.println(miProc.pid());		// Pid del proceso iniciado

		// Enlazar salida estandar del proceso hijo con este proceso padre
		InputStream mIs = miProc.getInputStream();
		InputStreamReader miIsr = new InputStreamReader(mIs);
		BufferedReader miIsrb = new BufferedReader(miIsr);
		String sInfo = "";
		try {
			while((sInfo = miIsrb.readLine()) != null)
			{
				System.out.println(sInfo);
			}
		} catch (IOException e) {
			e.printStackTrace();
		}
	}
}
```

Los procesos vuelven a ser los mismos que en el apartado anterior.

Método WaitFor con y sin tiempo

```
import java.io.File;
import java.io.IOException;

public class Tema1Prueba {
	public static void main (String [] args)
	{
		String [] sArg = new String [2];
		sArg[0] = "/usr/java/jdk-21.0.4/bin/java";		// Consultar localizacion con which java
		sArg[1] = "Tema1PruebaH";				// .class, sin indicar
		ProcessBuilder mipb = null;
		Process miProc = null;

		miProc = null;
		mipb = new ProcessBuilder(sArg);
		mipb.directory(new File("./bin"));

		try {
			miProc = mipb.start();		// Inicia el proceso
		} catch (IOException e) {
			e.printStackTrace();
		}

		System.out.println(miProc.pid());		// Pid del proceso iniciado
		int iSalida= 111;
		boolean bSalida = false;
		try {
			iSalida = miProc.waitFor();
			// bSalida = miProc.waitFor(1, TimeUnit.SECONDS);
			// bSalida = miProc.waitFor(4, TimeUnit.SECONDS);
		} catch (InterruptedException e) {
			e.printStackTrace();
		}

		if(bSalida)		// Si ha terminado pido el valor de retorno. Sino da excepcion
		{
			iSalida = miProc.exitValue();
		}
		// System.out.println(bSalida);
		System.out.println(iSalida);
	}
}

public class Tema1PruebaH {
	public static void main (String [] args)
	{
		try {
			Thread.sleep(3000);
		} catch (InterruptedException e) {
			e.printStackTrace();
		}
		 // System.exit(0);
		System.exit(10);

	}
}
```

Envío de información desde el proceso padre al hijo por su entrada estándar

```
import java.io.BufferedReader;
import java.io.File;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.io.OutputStreamWriter;
import java.io.Writer;

public class Tema1Prueba {
	public static void main (String [] args)
	{
		String [] sArg = new String [2];
		sArg[0] = "/usr/java/jdk-20.0.2/bin/java"; // Consultar localizacion con which java
		sArg[1] = "Tema1PruebaH";                                          // .class, sin indicar
		ProcessBuilder mipb = null;
		Process miProc = null;

		mipb = new ProcessBuilder(sArg);
		mipb.directory(new File("./bin"));

		try {
			miProc = mipb.start();          // Inicia el proceso
		} catch (IOException e) {
			e.printStackTrace();
		}

		System.out.println(miProc.pid());          // Pid del proceso iniciado


		// Enlazar entrada estandar del proceso hijo con este proceso padre
		Writer wr = new OutputStreamWriter(miProc.getOutputStream());
		try {
			wr.write("HOLA\n");
			wr.flush();
		} catch (IOException e) {
			// TODO Auto-generated catch block
			e.printStackTrace();
		}

		// Enlazar salida estandar del proceso hijo con este proceso padre
		InputStream mIs = miProc.getInputStream();
		InputStreamReader mIsr = new InputStreamReader(mIs);
		BufferedReader mIsrb = new BufferedReader(mIsr);
		String sInfo = "";
		try {
			while((sInfo = mIsrb.readLine()) != null)
			{
				System.out.println(sInfo);
			}
		} catch (IOException e) {
			e.printStackTrace();
		}
	}
}
```

```
import java.util.Scanner;
public class Tema1PruebaH {
	public static void main (String [] args)
	{
		Scanner miScan = new Scanner(System.in);
		String sInfo = miScan.nextLine();          // A la espera hasta recibir algo
		System.out.println("Recibido: " + sInfo);
		System.out.println("Te devuelvo el saludo");
		miScan.close();
	}
}
```

Redirigir entrada y salida estándar a ficheros

```
import java.io.File;
import java.io.IOException;

public class Tema1Prueba {
	public static void main (String [] args)
	{
		String [] sArg = new String [2];
		sArg[0] = "/usr/java/jdk-20.0.2/bin/java"; // Consultar localizacion con which java
		sArg[1] = "Tema1PruebaH";                                  // .class, sin indicar
		ProcessBuilder mipb = null;
		Process miProc = null;

		mipb = new ProcessBuilder(sArg);
		mipb.directory(new File("./bin"));

		File fInp = new File("numero.txt");
		File fOut = new File("salida.txt");
		File fErr = new File("error.txt");

		mipb.redirectInput(fInp);
		mipb.redirectOutput(fOut);
		mipb.redirectError(fErr);

		try {
			miProc = mipb.start();		// Inicia el proceso
		} catch (IOException e) {
			// TODO Auto-generated catch block
			e.printStackTrace();
		}

		System.out.println(miProc.pid());		// Pid del proceso iniciado
	}
}

import java.util.Scanner;

public class Tema1PruebaH {
	public static void main (String [] args)
	{
		int iValorUno,iValorDos,iResul;
		Scanner miScan = new Scanner(System.in);
		String sInfo = miScan.nextLine();
		iValorUno = Integer.parseInt(sInfo);
		sInfo = miScan.nextLine();
		iValorDos = Integer.parseInt(sInfo);

		iResul = iValorUno + iValorDos;
		System.out.println(iValorUno + " + " + iValorDos + " es igual a " + iResul);
		miScan.close();
	}
}
```

Fichero numero.txt en ruta del proyecto

5
3

Validar salida en ruta del proyecto. Ficheros salida.txt y error.txt

Para validar error.txt. Lanzar processBuilder con comando ls -laeevefef

Otros métodos de Process a probar

isAlive() // Devuelve un boolean. True para el proceso vivo.
destroy() // Comentar kill -9 desde el terminal.